
Analysing the Role of AI-Assisted Programming in Modern Software Development

IT 3003 – Advanced Programming Techniques

NAME – T.L.S.H. LIYANAGE
REGISTRATION NO- 2023S20062
Index No – s17412

Introduction

Artificial Intelligence, simply called AI, is now a common part of software development. Tools such as AI code generators, debugging helpers, and smart programming assistants are widely used to make coding faster and more efficient.

This report looks at how AI-assisted programming is used in modern software engineering, the benefits and challenges it brings, and shows a practical example through the AI- supported creation of a Student Learning System.

Following the assignment requirements, the report includes Task 1, which discusses the concept of AI- assisted programming and its impact on software development. Task 2 presents both the manually developed version (Version A) and the AI-assisted implementation (Version B), together with the UML diagram and a comparison of the two approaches. Task 3 provides a critical analysis of how AI influenced the development process, while Task 4 concludes the report by reflecting on the overall benefits and limitations of using AI as a programming assistant.

Task 1: Conceptual Understanding

1.1 What is AI-Assisted Programming?

Developers using artificial intelligence tools to help them write, debug, test and maintain code is called AI-Assisted Programming. Machine Learning algorithms and Large Language Models (LLM) are specifically used IN Artificial Intelligence. Instead of fully developing a project using AI, software developers use AI tools to assist in developing software applications to improve the code quality and handle repetitive tasks.

1.2Types of AI coding tools

Modern AI tools have evolved from simple syntax completion to fully aware development assistant. They can implement fully developed codes with only a simple explanation as well as

running test cases for software reliability. These tools can be categorized according to their functionalities.

- **Code Generation and Autocompletion:** In most cases line-level and multi-line autocompletion happens where when developer types the tool predicts the next few lines of code. The AI can also generate an entire block of code by writing a comment describing a function. Ex: Github Copilot, SuperMaven
- **Interactive Chat Assistants:** Instead of guessing the developer can paste code block in these tools and ask them to explain the logic or find errors. Ex: ChatGPT, Claude, Gemini
- **Code Refactoring and Optimization:** AI tools can restructure existing codes without changing the external behaviour or the expected output. They can suggest cleaner versions for repeated codes to make the code more readable and efficient. Ex: Claude, GitHub Copilot
- **Automated Test Generation:** AI tools can write unit and integration tests to check whether the code gives the required outputs. This increases the reliability of the project. Ex: GitHub Copilot

1.3 Advantages in Software Development

- **Reduction in time-to-market**
 - AI tools have compressed the software development life cycle by automating most of the procedures.
 - With the help of AI tools development teams can develop new products with high quality in a short period of time.
- **Increase the productivity of developers**
 - In the past if there were any syntax issues or formatting problems developers had to read documentation and divide their focus between syntax correction and logic design.

- With AI tools developers can focus on high level logic design and let AI handle the syntax and formatting issues.
- Less Cost and Manpower
 - Instead of hiring junior developers organizations can use AI agents to complete simple tasks that are repetitive and low risk which will reduce the manpower and costs for another set of employees.
- Higher Code Quality
 - AI tools can inspect manually written codes and find errors in validation or exception handling. Also, by doing unit and integration testing AI tools can improve the reliability of the code.

1.4 Risks and Limitations (eg: hallucinated code, dependency, security issues)

- Security flaws and vulnerabilities: AI tools do not understand security rules most of the time. They are simply algorithms that predict code patterns based on their trained data.
- Hallucinated Code Structures: AI tools sometimes use or make up tools that are believable, but they do not exist such as a new software package but in reality there is no such package.
- Over Dependency: When developers rely heavily on AI tools the developer may skip through code reviews since AI usually generate codes that work on the surface, but this may later cause bugs in production.
- Skill Decline: When developers use AI tools for every simple task and generate codes and test cases through AI they will have lack of skills and experience in debugging and logical thinking abilities.

Task 2: Practical Implementation

2.1 Selected Problem

The ticket booking management system is a digital platform designed for customers and administrators. For customers after registering in the system and login they can search for a event a book a ticket for that event and make secure payments through the system. If the customer want, they can also cancel the ticket through their profile. The system administrators are responsible for updating the events and managing reservations. They will check whether purchases tickets are available for a customer and manage their reservations. They are also responsible for managing the user accounts.

2.2 Solution Development

The Ticket Booking Management System was developed using two approaches.

- Version A: Traditional programming approach
- Version B: AI assisted programming approach

2.2.1 Version A

In the traditional programming approach the code was developed without the use of AI tools. All the classes and methods and the logic of the system was implemented without the assistance of AI tools considering the Java practices learned through the course.

Screenshots of the manually developed code is below.

```

import java.util.Scanner;

public class TicketBooking {

    Run | Debug
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println(x: "TICKET BOOKING MANAGEMENT SYSTEM");

        System.out.print(s: "Enter Customer ID: ");
        int customerId = input.nextInt();
        input.nextLine();

        System.out.print(s: "Enter Customer Name: ");
        String customerName = input.nextLine();

        System.out.print(s: "Enter Email: ");
        String customerEmail = input.nextLine();

        System.out.print(s: "Enter Password: ");
        String password = input.nextLine();

        System.out.print(s: "Enter Phone Number: ");
        String phone = input.nextLine();

        Customer customer = new Customer(customerId, customerName, customerEmail, password, phone);
        customer.register();
        customer.login();

        System.out.println();

        customer.searchEvent();

        Event event = new Event(
            eventId: 101,
            eventName: "Naadanu Concert",

```

Screenshot 1 – TicketBooking Class

```

        System.out.println(x: "\n---Event Details---");
        System.out.println("Event ID : " + event.getEventId());
        System.out.println("Event Name : " + event.getEventName());
        System.out.println("Date : " + event.getEventDate());
        System.out.println("Location : " + event.getEventLocation());
        System.out.println("Available Tickets : " + event.getAvailableTickets());
        System.out.println("Ticket Price : Rs." + event.getTicketPrice());

        System.out.print(s: "\nDo you want to book a ticket for this event? (yes/no): ");
        String answer = input.nextLine();

        if(answer.equalsIgnoreCase(anotherString: "yes")){

            System.out.print(s: "Enter Seat Number: ");
            String seat = input.nextLine();

            Ticket ticket = new Ticket(ticketId: 1001, seat, event);
            System.out.println(x: "\nTicket Details");
            System.out.println("Ticket ID : " + ticket.getTicketId());
            System.out.println("Seat Number : " + ticket.getSeatNo());
            Payment payment = new Payment(event.getTicketPrice());
            payment.makePayment();
            customer.bookTicket();

            System.out.print(s: "\nDo you want to cancel the ticket? (yes/no): ");
            String cancel = input.nextLine();

            if(cancel.equalsIgnoreCase(anotherString: "yes")){
                customer.cancelTicket();
            }
        }
    }
}

```

Screenshot 2 – TicketBooking Class

```

        if(answer.equalsIgnoreCase(anotherString: "yes")){

            System.out.print(s: "Enter Seat Number: ");
            String seat = input.nextLine();

            Ticket ticket = new Ticket(ticketId: 1001, seat, event);
            System.out.println(x: "\nTicket Details");
            System.out.println("Ticket ID : " + ticket.getTicketId());
            System.out.println("Seat Number : " + ticket.getSeatNo());
            Payment payment = new Payment(event.getTicketPrice());
            payment.makePayment();
            customer.bookTicket();

            System.out.print(s: "\nDo you want to cancel the ticket? (yes/no): ");
            String cancel = input.nextLine();

            if(cancel.equalsIgnoreCase(anotherString: "yes")){
                customer.cancelTicket();
            }
            else{
                System.out.println(x: "Booking Cancelled.");
            }
            System.out.println(x: "Thank you for booking with us.");
        }
    }
}

```

Screenshot 3 – TicketBooking Class

```

TICKET BOOKING MANAGEMENT SYSTEM
Enter Customer ID: 101
Enter Customer Name: Anne
Enter Email: anne2002@gmail.com
Enter Password: 245683
Enter Phone Number: 04134568
User Anne registered
User Anne logged in

Searching for an event

----Event Details----
Event ID : 101
Event Name : Naadanu Concert
Date : 20/08/2026
Location : Nelum Pokuna Theatre
Available Tickets : 100
Ticket Price : Rs.3500.0

Do you want to book a ticket for this event? (yes/no): yes
Enter Seat Number: A3
Enter Number of Tickets: 1

Ticket Details
Ticket ID : 1001
Seat Number : A3

```

Screenshot 4 – Traditional Version Output

```
Available Tickets : 100
Ticket Price : Rs.3500.0

Do you want to book a ticket for this event? (yes/no): yes
Enter Seat Number: A3
Enter Number of Tickets: 1

Ticket Details
Ticket ID : 1001
Seat Number : A3
Payment of amount 3500.0 made successfully
Ticket booked successfully

Do you want to cancel the ticket? (yes/no): no
Thank you for booking with us.
```

Screenshot 5 - Traditional Version Output

2.2.2 Version B

In version B the code was modified with the help of AI tool GitHub Copilot. With input validation and exception handling as well as increased readability through reducing repetitive code structures the code is modified to work efficiently.

AI prompts used in developing the AI version

- Improve the code using data validation. Ticket price ticket amount cannot be negative or zero.
- The mobile number should not exceed 10 digits and validate all the user inputs.
- Without changing the overall functionality improve the code structure according to OOP concepts
- Improve the main class by reducing repetitions and increase readability but keep the functions as it is
- Refactor classes to improve encapsulation without changing the behavior.

Screenshots of the code developed using AI tools is below.


```

import java.util.Scanner;

public class TicketBooking {

    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println(x: "TICKET BOOKING MANAGEMENT SYSTEM");

        try {
            Customer customer = createCustomer(input);
            initializeCustomer(customer);

            Event event = createEvent();
            displayEvent(event);

            if (confirm(input, prompt: "\nDo you want to book a ticket for this event? (yes/no): ")) {
                processBooking(input, customer, event);
            } else {
                System.out.println(x: "Booking Cancelled.");
            }
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid input: " + e.getMessage());
        } finally {
            input.close();
        }

        System.out.println(x: "Thank you for booking with us.");
    }

    private static Customer createCustomer(Scanner input) {
        int customerId = readPositiveInt(input, prompt: "Enter Customer ID: ");
        String customerName = readCustomerName(input, prompt: "Enter Customer Name: ");
        String customerEmail = readEmail(input, prompt: "Enter Email: ");
        String password = readPassword(input, prompt: "Enter Password: ");
        String phone = readPhoneNumber(input, prompt: "Enter Phone Number: ");
        return new Customer(customerId, customerName, customerEmail, password, phone);
    }

    private static Event createEvent() {
        return new Event(
            eventId: 101,
            eventName: "Naadanu Concert",

```

Screenshot 6 – TicketBooking Class (AI Version)

```

public class TicketBooking {

    private static Event createEvent() {
        return new Event(
            eventId: 101,
            eventName: "Naadanu Concert",
            eventDate: "20/08/2026",
            eventLocation: "Nelum Pokuna Theatre",
            availableTickets: 100,
            ticketPrice: 3500.00);
    }

    private static void initializeCustomer(Customer customer) {
        customer.register();
        customer.login();
        System.out.println();
        customer.searchEvent();
    }

    private static void displayEvent(Event event) {
        System.out.println(x: "\n----Event Details----");
        event.displayEventDetails();
    }

    private static boolean confirm(Scanner input, String prompt) {
        return readYesNo(input, prompt).equalsIgnoreCase(anotherString: "yes");
    }

    private static void processBooking(Scanner input, Customer customer, Event event) {
        String seat = readSeatNumber(input, prompt: "Enter Seat Number: ");
        int ticketAmount = readPositiveInt(input, prompt: "Enter Number of Tickets: ");

        Ticket ticket = new Ticket(ticketId: 1001, seat, event);
        System.out.println(x: "\nTicket Details");
        ticket.displayTicketDetails();

        double totalAmount = event.calculateTotalCost(ticketAmount);
        Payment payment = new Payment(totalAmount);
        payment.makePayment();
        customer.bookTicket();

        if (confirm(input, prompt: "\nDo you want to cancel the ticket? (yes/no): ")) {
            customer.cancelTicket();
        }
    }
}

```

Screenshot 7 – TicketBooking Class (AI Version)

```

        if (confirm(input, prompt: "\nDo you want to cancel the ticket? (yes/no): ")) {
            customer.cancelTicket();
        }
    }

    private static int readPositiveInt(Scanner input, String prompt) {
        while (true) {
            System.out.print(prompt);
            if (input.hasNextLine()) {
                throw new IllegalArgumentException(s: "Input ended unexpectedly.");
            }
            String value = input.nextLine().trim();
            try {
                int number = Integer.parseInt(value);
                if (number > 0) {
                    return number;
                }
                System.out.println(x: "Please enter a value greater than zero.");
            } catch (NumberFormatException e) {
                System.out.println(x: "Please enter a valid whole number.");
            }
        }
    }

    private static String readRequiredText(Scanner input, String prompt) {
        while (true) {
            System.out.print(prompt);
            if (input.hasNextLine()) {
                throw new IllegalArgumentException(s: "Input ended unexpectedly.");
            }
            String value = input.nextLine().trim();
            if (!value.isEmpty()) {
                return value;
            }
            System.out.println(x: "This field cannot be empty.");
        }
    }

    private static String readCustomerName(Scanner input, String prompt) {
        while (true) {
            String name = readRequiredText(input, prompt);

```

Screenshot 8 – TicketBooking Class (AI Version)

```

        }
        String value = input.nextLine().trim();
        if (!value.isEmpty()) {
            return value;
        }
        System.out.println(x: "This field cannot be empty.");
    }

    private static String readCustomerName(Scanner input, String prompt) {
        while (true) {
            String name = readRequiredText(input, prompt);
            if (name.matches(regex: "[A-Za-z\\s]+")) {
                return name;
            }
            System.out.println(x: "User name should contain only letters and spaces.");
        }
    }

    private static String readSeatNumber(Scanner input, String prompt) {
        while (true) {
            String seat = readRequiredText(input, prompt);
            if (seat.matches(regex: "[A-Za-z0-9-]+")) {
                return seat;
            }
            System.out.println(x: "Seat number should be letters, numbers, or dashes only.");
        }
    }

    private static String readEmail(Scanner input, String prompt) {
        while (true) {
            String email = readRequiredText(input, prompt);
            if (email.contains(s: "@") && email.contains(s: ".")) {
                return email;
            }
            System.out.println(x: "Please enter a valid email address.");
        }
    }

    private static String readPassword(Scanner input, String prompt) {
        while (true) {
            String password = readRequiredText(input, prompt);

```

Screenshot 9 – TicketBooking Class (AI Version)

```

private static String readPassword(Scanner input, String prompt) {
    while (true) {
        String password = readRequiredText(input, prompt);
        if (password.length() >= 6) {
            return password;
        }
        System.out.println(x: "Password must be at least 6 characters long.");
    }
}

private static String readPhoneNumber(Scanner input, String prompt) {
    while (true) {
        String phone = readRequiredText(input, prompt);
        if (phone.matches(regex: "\\d{1,10}")) {
            return phone;
        }
        System.out.println(x: "Phone number must contain only digits and not exceed 10 digits.");
    }
}

private static String readYesNo(Scanner input, String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = input.nextLine().trim();
        if (value.equalsIgnoreCase(anotherString: "yes") || value.equalsIgnoreCase(anotherString: "no")) {
            return value;
        }
        System.out.println(x: "Please enter either 'yes' or 'no'.");
    }
}

```

Screenshot 10 – TicketBooking Class (AI Version)

```
TICKET BOOKING MANAGEMENT SYSTEM
Enter Customer ID: 0
Please enter a value greater than zero.
Enter Customer ID: 1
Enter Customer Name: Supuni
Enter Email: 202930
Please enter a valid email address.
Enter Email: supuni2002@gmail.com
Enter Password: 23
Password must be at least 6 characters long.
Enter Password: 123455
Enter Phone Number: 07167839322
Phone number must contain only digits and not exceed 10 digits.
Enter Phone Number: 0718345678
User Supuni registered
User Supuni logged in

Searching for an event

----Event Details----
Event ID : 101
Event Name : Naadanu Concert
Date : 20/08/2026
Location : Nelum Pokuna Theatre
Available Tickets : 100
Ticket Price : Rs.3500.0

Do you want to book a ticket for this event? (yes/no): yes
Enter Seat Number: A5
Enter Number of Tickets: 3

Ticket Details
Ticket ID : 1001
Seat Number : A5
Payment of amount 10500.0 made successfully
Ticket booked successfully

Do you want to cancel the ticket? (yes/no): yes
Ticket cancelled successfully
```

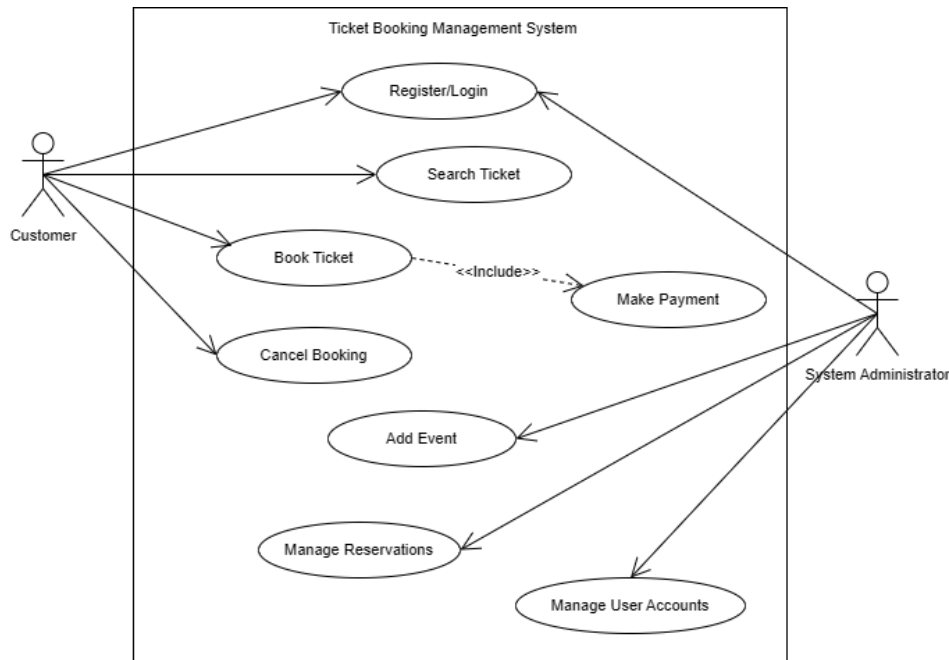
Screenshot 11 – AI Version Output

2.3 Comparison between Version A and Version B

Quality	Version A – Manual Coding	Version B – AI tool
Development time	For this development less time was required because most logic is on the TicketBooking. Java file. Simple and direct implementation.	Since there were validation methods and better class encapsulation, took more time. But more structured and reusable.
Code Quality	User inputs are taken without data validations. Classes are mutable and public getters are used for everything.	Higher quality compared to manual coding. Validation is encapsulated in constructors.
Error Handling	No error handling done. Input is read directly without any exception handling.	Throw meaningful exception handling messages.
Readability & Maintainability	Readable but hard to maintain. Future changes require large procedural flow.	more readable and easier to modify without changing the entire flow.

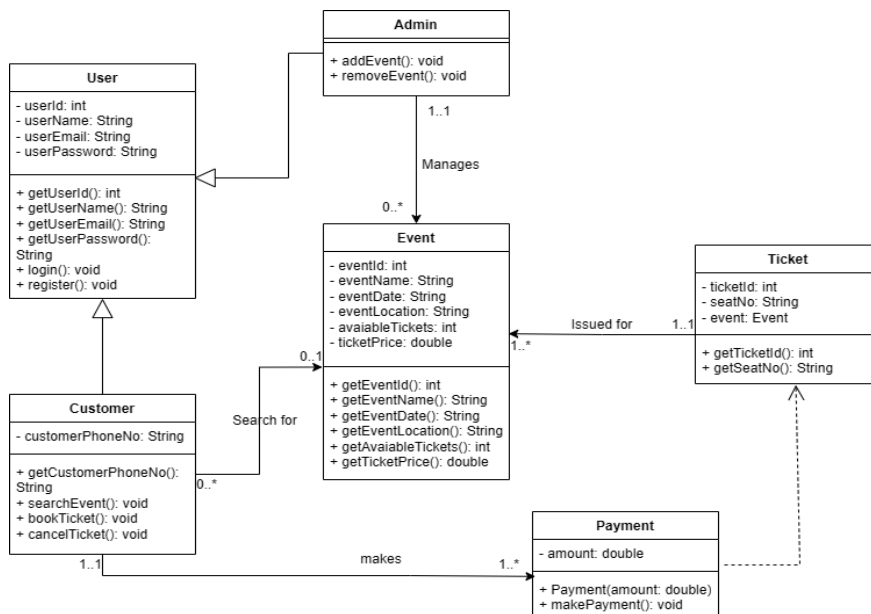
2.4 UML Diagrams

2.4.1 Use Case Diagram



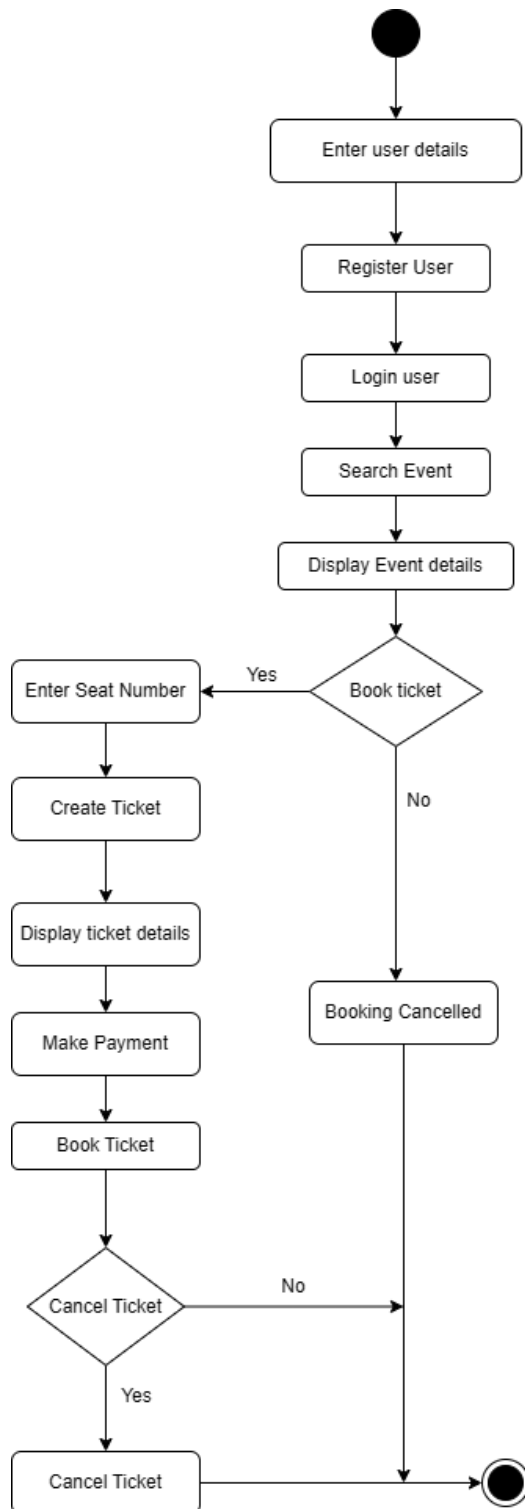
Use Case Diagram – Ticket Booking Management System

2.4.2 Class Diagram



Class Diagram – Ticket Booking Management System

2.4.3 Activity Diagram



Activity Diagram – Ticket Booking Management System

Taks 3: Critical Analysis

3.1 How AI changed the coding approach

When manually developing the system, the focus was mainly on writing syntax rather than the logic of the system. For choice preferences there were repetitive code lines and long code structures. Constantly formatting the code lines to breaking down features line by line the manual approach was time consuming and harder to develop.

In the AI approach rather than focusing on the syntax and formatting the system was developed focusing on the logic of the solution. With structural prompts AI version created a high-level product with less code duplications and more structured classes and methods using the core OOP concepts.

3.2 Whether AI improved or reduced the understanding

Improved Understanding: For complex integrations AI tools were very useful. With cleaner validation logic the AI tools were useful in handling input mismatches and various exceptions that were not properly handled through manual development.

Reduced Understanding: Since AI tools instantly generate complex logic and, on the surface, they give the expected output, instead of understanding the logic of the code lines in the AI version some functions were used. But when prompted the AI tools helped to identify the logic of these functions that were out of scope in the module.

3.3 In Which Cases should AI not be Used

- In high staked fields: Using AI tools without human verification in fields such as legal drafting or medical diagnostics can lead to legal, financial or safety issues, since AI tools operate on probability and patterns.

- When learning concepts and mastering skills: Using AI at the start of the learning process will diminish skills such as problem solving and critical thinking which are must skills for a developer.
- When handling private or confidential data: Uploading sensitive information to AI tools often risk in cyber-attacks and privacy violations.
- For real time facts and news: Relying on AI to get current news is highly unreliable. AI is trained based on past data thus it is not reliable to use AI to get current information.

3.4 Ethical considerations in AI generated code

Accountability: Consider a payment gateway generated using AI. If there are any issues in the payment processing or updating the responsibility of that bug will be on the developer. Even if the developer used AI for the task, it should be their responsibility to debug and check for any errors in the code.

Plagiarism: AI models are trained on public repositories thus sometimes AI may use copyrighted code or open-source code without giving proper credits. The developer must ensure that these codes do not violate software license or take uncredited work of others.

Data Privacy: Typing user information or database structures or source codes to AI tools will result in cyber attacks from other parties since these data are sent to external servers. Developers must be able to protect user privacy by not using sensitive information in AI tools.

Skill reduction: Heavy reliance on AI tools has a huge effect on the critical thinking and problem-solving skills of developers. The organizations must ensure that developer use AI tools to assist in the development rather than letting it do the full development.

Taks 4: Conclusion

In the educational sector AI can be used as a mentor available 24/7 acting as a learning growth asset. Instead of fully depending on AI using, it as a way of learning will improve the growth of any student. It can explain compilation errors or any unidentifiable error give insights into handling exceptions and validating data in a secure way. Also, AI can assist in learning complex logical designs and give experience on industry level coding structures.

In the industry AI is already used as a way of creating more efficient or optimized products. With correct protocols and guidelines developers can use AI tools to hone their skills as well as create products with minimum manual labour but high reliability. Instead of mechanical typing developers focus more on designing high level logical systems with strong security systems and financial value.

Through this project with the development of the same solution through different versions, it can be identified that using AI tools do not have a negative impact on the learning curve of a developer. If used alongside the developer's knowledge and skills AI tools are an asset to modern software development. AI is not a replacement for software developers it is a helper that will enhance their skills and develop their cognitive thinking.

Repository Link - <https://github.com/SHiranya2002/Individual-Report.git>